

Introduction SPRING



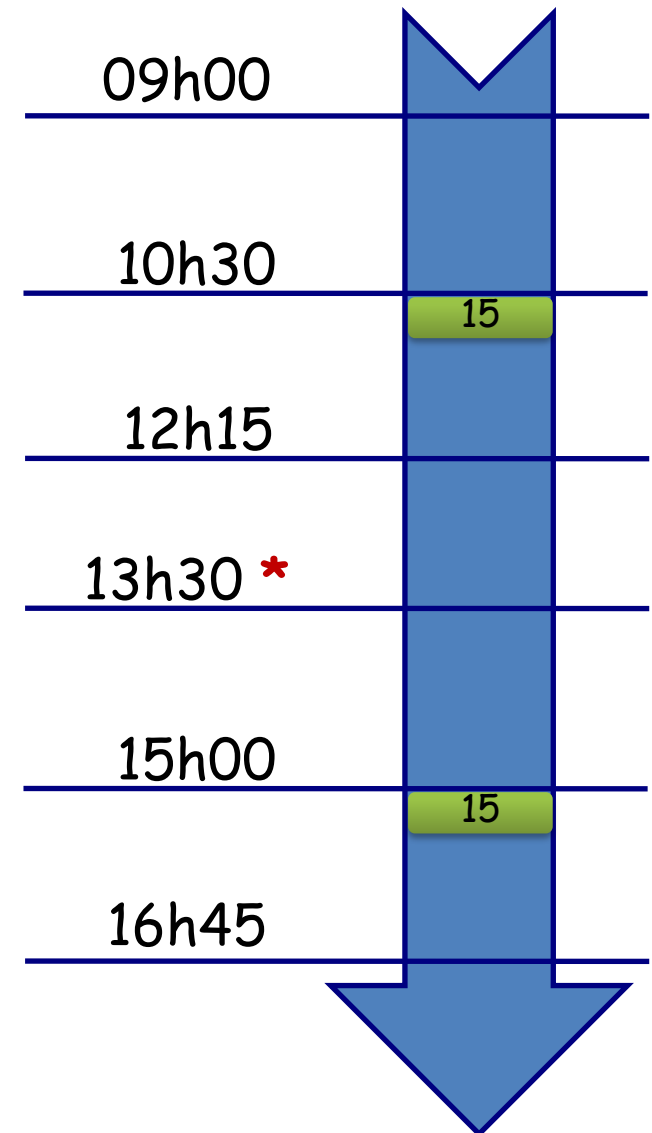
Bureaux E204 | E304

Plan du Cours

- Introduction : Horaires, Evaluation
- Spring et le Marché de l'Emploi
- Contenu du Module **Spring**
- Historique Spring
- Concurrents Spring
- projets Spring
- Framework Spring
- Présentation et Installation des Outils : JDK, IntelliJ, Tomcat, MySQL, Maven, Log4J,

Horaires

- Durée Totale : **42 heures**
 - Séances : **14 séances**
 - Cours : **12 heures**
 - TP : **30 heures**
-
- Durée de chaque Séance : **3 heures**
 - Durée de la Pause : **15 minutes**
-
- * Vendredi : **13h45**



Evaluation

- L'évaluation se fait tout au long du module, et non pas uniquement à la fin.
- La moyenne du module est calculée comme suit :
Moyenne = Note Contrôle Continu * 40% + Note Examen * 60%
- Le **Contrôle Continu** prend en compte l'Assiduité, la Participation, les **TP** à faire en cours ou chez vous et aussi un **Examen Blanc Pratique**.
- L'**Examen** sera pratique.

Evaluation

- Exemple d'évaluation :

Note Participation (/6)	Homeworks (/12)	Bonus (/2)	Moy	Moy VF
4,222222222	11,5	2	17,72222222	18
6	11	2	19	19
5,111111111	10	2	17,11111111	17,5
0,888888889	6	1	7,88888889	8
1,777777778	7	1	9,77777778	10
0	10	0	10	10
6	12	2	20	20

Spring et le Marché du Travail

- Les offres d'emploi « **Cherche Développeur Java** », en général cherchent un développeur dans les technologies suivantes :
- **Connaissances Techniques requises :**
 - Maîtrise des langages et Framework: Angular (idéalement 12+), API REST, Java, Spring Framework (Spring Boot, Spring Core, Spring Data, Spring AOP, Spring Security);
 - Maîtrise du HTML5, CSS ;
 - Connaissance du JavaScript et jQuery ;
 - Connaissance du Git et du Eclipse ;
 - Connaissance minimale du JPA et de Hibernate ;
 - Connaissance basique du SQL ;
 - Connaissance en micro services est un atout.
- <https://www.tanitjobs.com/job/1453753/d%C3%A9veloppeur-fullstack-angular-springboot-java/?backPage=1&searchID=1718024332.5001>

Contenu du Module Spring

- ❑ **Introduction à Spring et Mise en place de l'Environnement:** Fournir une compréhension de base de Spring et mettre en place l'environnement de développement avec JDK 17, Spring Boot, et Maven.
- ❑ **Spring Boot Maven:** Comprendre les bases de Spring Boot et l'utilisation de Maven pour gérer les dépendances et le cycle de vie du projet
- ❑ **Spring Data JPA:** Apprendre à utiliser Spring Data JPA pour gérer les interactions avec la base de données.
- ❑ **Lombok:** Simplifier le code Java en utilisant Lombok pour générer automatiquement des constructeurs, getters, setters, et autres.
- ❑ **IoC & DI (Injection des Dépendances) :** Comprendre le concept d'Inversion de Contrôle et l'Injection de Dépendances dans Spring.
- ❑ **Spring MVC REST :** Créer des services RESTful avec Spring MVC et tester avec Postman.

- **TP étude de cas** Appliquer les connaissances acquises pour développer une application complète
- **Spring Scheduler** Planifier des tâches avec le module Spring Scheduler.
- **Spring Batch** Comprendre et utiliser Spring Batch pour le traitement par lots.
- **AOP** Apprendre à utiliser la programmation orientée aspects pour séparer les préoccupations transversales.
- **Examen Blanc Pratique** (Spring Boot - REST - Spring Data – AOP-etc..)

Bref Historique JavaEE

- Java Platform, Enterprise Edition (Java EE), maintenant rebaptisé Jakarta EE, a été introduit par Sun Microsystems en 1998 pour fournir un ensemble d'API et d'environnements d'exécution nécessaires au développement d'applications d'entreprise robustes, évolutives et sécurisées. Java EE a été conçu pour simplifier le développement de solutions d'entreprise en fournissant une infrastructure standardisée et modulaire, incluant des technologies comme Servlets, JSP (JavaServer Pages), EJB (Enterprise JavaBeans), et JPA (Java Persistence API).

Bref Historique JavaEE

Avant l'avènement des Enterprise Java Beans (EJB), les développeurs Java utilisaient les JavaBeans pour créer des applications Web, mais ceux-ci ne pouvaient pas gérer les transactions et la sécurité nécessaire pour les applications d'entreprise robustes. L'arrivée des EJB a apporté ces services indispensables, mais leur complexité de mise en œuvre a poussé les développeurs à chercher des solutions plus simples pour le développement d'applications d'entreprise.

Historique Spring

- **2002** : **Rod Johnson** publie son livre «Expert One-on-One J2EE Design and Development», dans lequel il propose du code, qui va devenir plus tard le Framework Spring
- **2004** : Rod Johnson publie son livre «**J2EE Development without EJB**».
- **2004** : Spring 1.0, licence Apache 2.0
- **2005** : Spring devient populaire, en particulier en réaction par rapport aux EJB 2.x

=> **La configuration de l'application par la main rendait la manipulation de Spring difficile.**

- **2006** : Spring 2.0 (Introduction de l'injection de dépendances)
- **2007** : Spring 2.5, avec support des **annotations**
- **2009** : Spring 3.0
- **2013** : Spring 4.0
- **2017** : Spring 5.x,
- **Juillet 2022** : Spring 5.3.22 (version actuelle)

Concurrents Spring

Jakarta EE

vs.

Spring

vs.

Quarkus



Concurrents Spring

Jakarta EE

- Jakarta EE 8, comme son nom l'indique, est fonctionnellement identique à Java EE 8, publié par Oracle et possède les mêmes spécifications.
- Jakarta EE a été optimisé pour le **cloud computing**.
- Selon la fondation Eclipse, l'un des principaux objectifs de Jakarta EE est d'accélérer le développement d'applications d'entreprise pour le Cloud Computing (les applications cloud natives).



Concurrents Spring

Jakarta EE

- La plateforme **JakartaEE** est en pleine mutation et modification. Attendons qu'elle soit stable.
- La plateforme **JavaEE** est un ensemble de spécifications **JSF, EJB, JPA**. Elle n'est plus maintenue par Oracle, donc bientôt obsolète.
- Le cœur de cette plateforme est les EJB.
- Inconvénients des EJB :
 - Difficile à coder, il faut implémenter des interfaces spécifiques ...
 - Les tests unitaires sont difficiles à réaliser.
 - C'est une solution qui nécessite un serveur d'application ⇒ lourd et gourmand en ressources.

Concurrents Spring

Jakarta EE vs Spring

- Choisir entre Jakarta EE et Spring dépend principalement des préférences et des besoins spécifiques du projet. Jakarta EE, basé sur des standards ouverts et géré par une communauté, offre une large gamme de spécifications pour le développement d'applications d'entreprise. Ses défenseurs soulignent la continuité et l'innovation soutenue par une gouvernance communautaire, garantissant ainsi une longévité et une évolution progressive des technologies. En revanche, Spring se distingue par sa simplicité, sa flexibilité et sa grande popularité sur le marché. Reconnu pour sa fiabilité à long terme et sa capacité à s'adapter rapidement aux nouvelles exigences, Spring attire les développeurs à la recherche d'une solution éprouvée et bien soutenue. Le choix se résume souvent à la préférence pour une gouvernance communautaire robuste et ouverte avec Jakarta EE ou à une adoption rapide et à une flexibilité accrue offertes par Spring, adaptées aux besoins spécifiques en matière de développement et d'évolutivité des applications d'entreprise.

Concurrents Spring

Quarkus

Quarkus est un framework Java natif Kubernetes conçu pour les machines virtuelles Java (JVM) optimisant Java spécifiquement pour les conteneurs et lui permettant de devenir une plate-forme efficace pour les environnements sans serveur, cloud et Kubernetes.

Tout comme Spring, Quarkus a été conçu pour être facile à utiliser dès le départ, avec des fonctionnalités avec peu ou pas de configuration.



Concurrents Spring

Quarkus vs. Spring

Spring et Quarkus sont en concurrence directe sur plusieurs critères clés tels que le temps de démarrage et la consommation de mémoire. Spring, avec sa large adoption et sa maturité, offre une gamme complète de fonctionnalités pour le développement d'applications d'entreprise, mais peut être critiqué pour son temps de démarrage relativement plus long et une consommation de mémoire plus élevée.

En revanche, Quarkus se positionne comme une alternative moderne, optimisée pour les architectures cloud-native et les microservices, offrant des temps de démarrage ultra-rapides et une empreinte mémoire réduite. Les développeurs se tournent souvent vers Quarkus pour sa capacité à répondre aux exigences d'évolutivité et d'architecture sans serveur, en offrant une meilleure efficacité opérationnelle et des performances améliorées dans les environnements de production modernes.

Le choix entre Spring et Quarkus dépendra ainsi des priorités spécifiques du projet, que ce soit une large gamme de fonctionnalités avec Spring ou une performance optimale et une efficacité opérationnelle avec Quarkus, adaptées aux besoins de développement actuels.

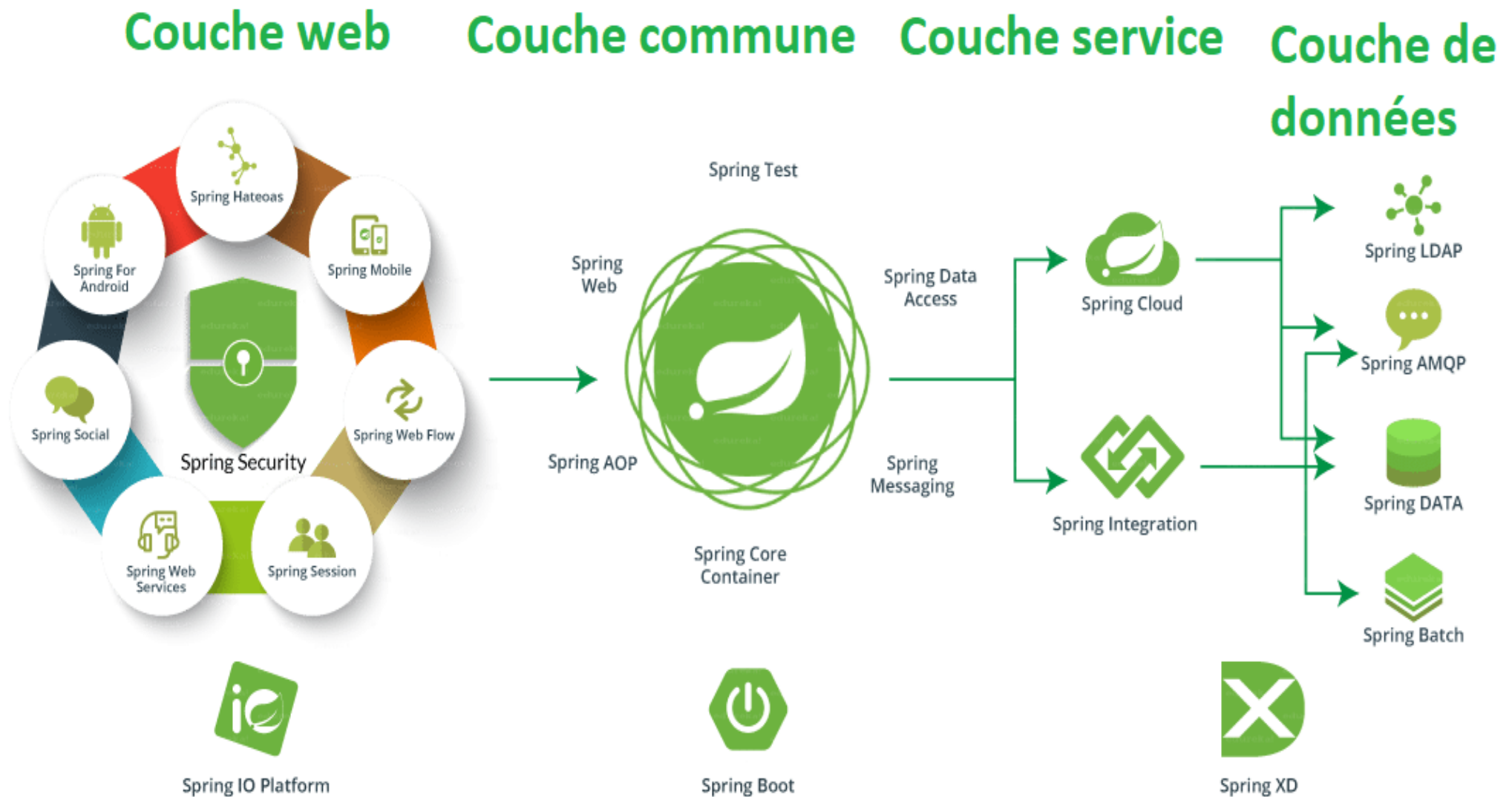
Concurrents Spring

Basé sur la comparaison et la demande du marché, nous allons nous focaliser dans notre cours sur **Spring** et utiliser une implémentation de JPA fournie par Spring : **Spring Data JPA**.

Les Projets Spring

- <https://spring.io/projects> :
- **Spring est un ensemble de projets.**
- Spring a une vaste communauté et une bonne documentation.
- Le code est sous licence Apache 2, il est Open Source et disponible sur GitHub.

Les Projets Spring



Les Projets Spring

- **Spring Framework** : Le cœur de l'écosystème Spring, il fournit une infrastructure complète pour le développement d'applications Java, incluant l'inversion de contrôle (IoC), l'injection de dépendances, et des modules pour la gestion des transactions, la sécurité, etc.
- **Spring Boot** : Un projet permettant de créer des applications Spring autonomes et prêtes à l'emploi avec une configuration minimale. Il simplifie le développement en intégrant automatiquement les dépendances et en configurant l'application selon les meilleures pratiques.
- **Spring Data** : Facilite l'accès aux données en fournissant un modèle de programmation cohérent pour la gestion des accès aux données, avec des modules pour MongoDB, Redis, JDBC, JPA, etc.
- **Spring Security** : Un cadre puissant pour l'authentification et la gestion des autorisations dans les applications Spring, offrant une sécurité robuste et personnalisable.
- **Spring Cloud** : Offre des outils pour le développement d'applications distribuées et basées sur les microservices, incluant la configuration externe, la découverte de services, la gestion de la tolérance aux pannes, etc.
- **Spring Batch** : Un cadre pour le traitement par lots (batch processing) dans les applications Java, permettant le traitement de grands volumes de données avec des transactions et une gestion des erreurs.
- **Spring Integration** : Facilite l'intégration d'applications via des modèles de programmation basés sur des flux (messaging patterns), supportant les EAI (Enterprise Application Integration) et les workflows.
- **Spring Web Services** : Facilite le développement d'applications web basées sur des services web SOAP et RESTful en utilisant Spring.
- **Spring MVC** : Un framework léger pour le développement d'applications web basées sur le modèle-vue-contrôleur (MVC), intégré dans Spring Framework.

Framework Spring (Spring Core)

- C'est quoi un **Framework** :
 - C'est un **cadre de développement**
 - Contient des «bonnes pratiques»
 - Permet d'éviter de recoder des classes utilitaires
 - Permet de se focaliser sur le métier
- Spring fournit la «plomberie» : le socle technique
- Les développeurs peuvent se concentrer sur le code métier (le vrai travail)
- « Attention un **framework** ne doit pas être considéré comme une **plate-forme**, dans la mesure où il n'intègre pas d'environnement d'exécution système ou applicatif. »

Framework Spring



Framework Spring

- Le Framework Spring est un socle pour le développement d'applications.
- Il fournit de nombreuses fonctionnalités.
- C'est l'un des Frameworks les plus répandus dans le monde Java.
- Framework Open Source
- **Framework Spring (Spring Core)** : <http://projects.spring.io/spring-framework>

Framework Spring

Principes Fondamentaux de l'Écosystème Spring :

- **Inversion de Contrôle (IoC)** : Gestion des dépendances par le framework, réduisant la complexité du code.
- **Programmation Orientée Aspect (AOP)** : Séparation des préoccupations transversales (logging, transactions, sécurité).
- **Configuration Simplifiée** : Utilisation d'annotations et de conventions pour simplifier la configuration.
- **Modules Riches** : Propose une variété de modules pour les différentes couches de l'application (Spring MVC, Spring Data, Spring Security, etc.).

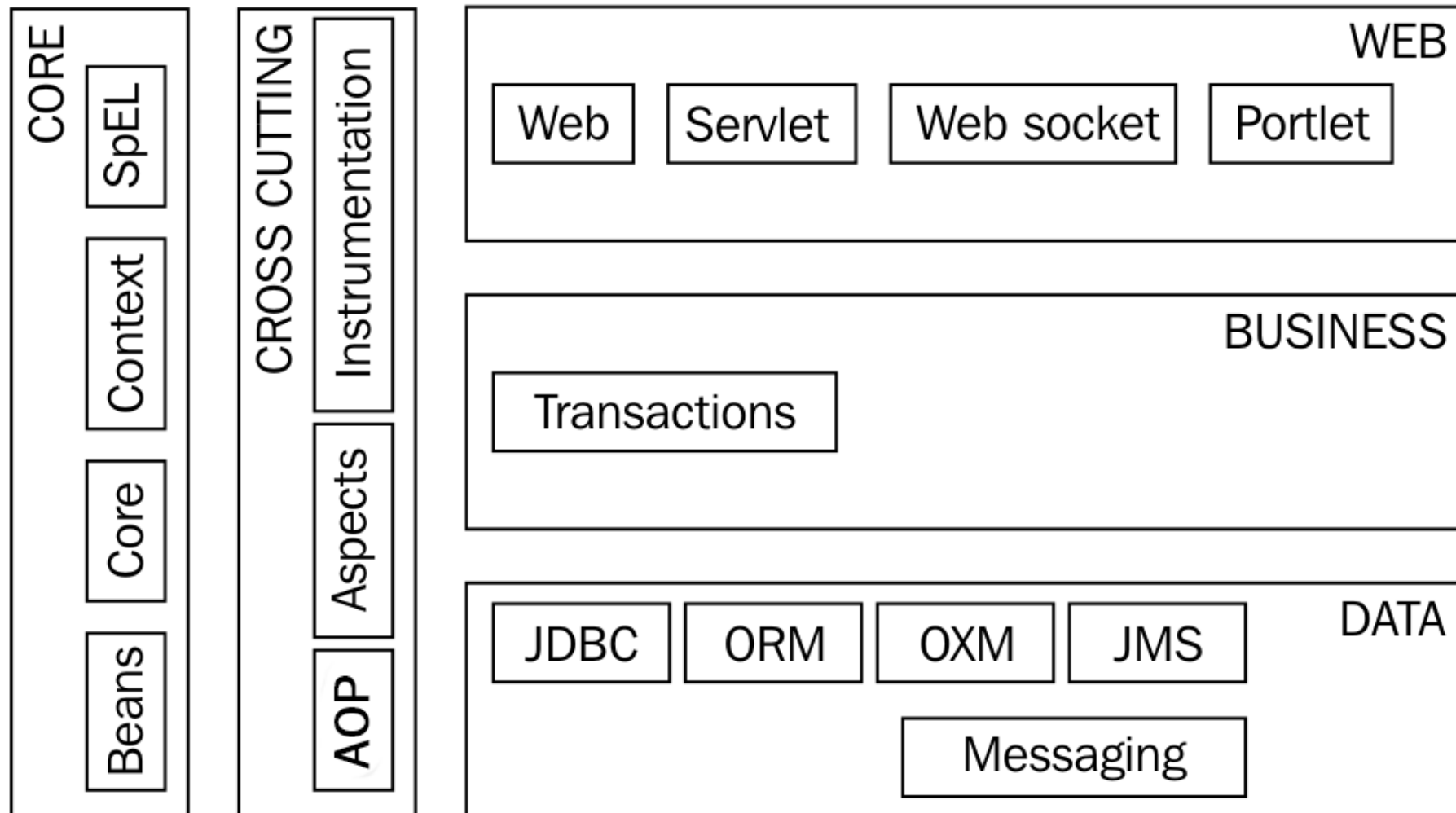
Apports Fonctionnels de Spring dans les Projets Java :

- **Réduction du Code Boilerplate** : Moins de code standard à écrire, ce qui accélère le développement.
- **Testabilité Améliorée** : Facilite l'écriture de tests unitaires et d'intégration.
- **Extensibilité et Modularity** : Permet de développer des applications modulaires et évolutives.
- **Support pour les Applications Entreprise** : Intégration avec les technologies d'entreprise comme JMS, JPA, et les services web

Framework Spring

- Le but de Spring est de faciliter et de rendre productif le développement d'applications.
- Il fournit beaucoup de fonctionnalités :
 - **Injection de Dépendances** (IoC : Inversion de Control).
 - **AOP** : Aspect Oriented Programming : Injection de Code en RunTime.
 - **Data Access** : Permet de simplifier l'accès aux données (DAO, ORM, Transaction, ...).
 - **Web** : Permet de développer des interfaces web évoluées (MVC).
 - **Testabilité** de l'application facilitée
 - **Intégration** : Spring offre un ESB, qui permet l'intégration et la communication entre les applications (EAI).

Framework Spring



Important

- Spring Boot 2.0 était la première version de la ligne 2.x et a été publiée le 28 février 2018. L'équipe Spring vient de publier Spring Boot 2.7, ce qui signifie qu'ils maintiennent la ligne 2.x depuis un peu plus de 4 ans.
- **Cette révision majeure** sera basée sur **Spring Framework 6.0 et nécessitera Java 17 ou une version plus récente**. Ce sera également la première version de Spring Boot à utiliser les API Jakarta EE 9 (Jakarta.*) au lieu de EE 8 (Javax.*).
- Spring Boot 3 est conçu pour être compatible avec les versions modernes de Java, y compris JDK 17. Cela garantit que vous pouvez utiliser les fonctionnalités les plus récentes de Java tout en bénéficiant de l'écosystème riche de Spring Boot.

Prochain Cours

- **Accédez à Spring Initializr :**

Rendez-vous sur le site de [Spring Initializr](https://start.spring.io).

- **Configurer les détails du projet :**

- **Project:** Choisissez Maven Project ou Gradle Project selon votre préférence.

- **Language:** Sélectionnez Java.

- **Spring Boot Version:** Choisissez 3.0.0 ou une version supérieure.

- **Project Metadata:**

- **Group:** com.example (ou tout autre nom de groupe que vous préférez).

- **Artifact:** my-spring-boot-app (nom de votre application).

- **Name:** my-spring-boot-app (nom de votre projet).

- **Description:** Demo project for Spring Boot 3 and JDK 17.

- **Package Name:** com.example.myspringbootapp (structure de package pour votre application).

- **Packaging:** Jar (ou War selon vos besoins).

- **Java:** 17 (ou sélectionnez la version JDK 17 si disponible).

- Finaliser l'Installation de l'Environnement de travail (JDK, INTELLIJ, MySQL, Maven)

- Comprendre **Maven**

- Créer vos premiers projets avec les outils ci-dessus.

INTRODUCTION SPRING

Si vous avez des questions, n'hésitez pas à nous contacter :

Département Informatique
UP Architectures des Systèmes d'Information

Bureaux E204 | E304

Introduction SPRING

